VS-03 Data Editing and Enrichment

Table of contents

VS-03.0 Data Editing and Enrichment . . . 2
- Purpose and Scope . . . 2
- Position in the Statistical Value Chain . . . 2
- Steady State Transition . . . 3
- The Four-Process Governance Pattern . . . 4
- Governing Principles . . . 4
- Capability Foundations . . . 5
- Metadata Perspective . . . 6

Business Roles and Actors . . . 6
Design Processes — "provides metadata" . . . 7
- Design Error-Detection and Validation Rules . . . 7
- Design Imputation Methodology . . . 7
- Design Derivation and Coding Specifications . . . 8
- Design Integration and Linkage Rules . . . 8
- Design Quality Evaluation, Versioning, and Audit Framework . . . 9

Implementation Processes — "provides the execution process" . . . 9
- Build Error-Detection and Editing Engines . . . 9
- Configure Imputation and Coding Services . . . 11
- Build Derivation, Integration, and Versioning Pipelines . . . 11
- Test and Validate the Pipeline . . . 12

Execution Processes — "provides the statistical data" . . . 13
- Conduct Error Detection and Quality Diagnostics . . . 13
- Perform Data Editing and Correction . . . 13
- Impute Missing or Non-Response Data . . . 14
- Derive Additional Variables and Indicators . . . 15
- Perform Quality Evaluation and Versioning . . . 15

Management Processes — "provides quality reports" . . . 16
- Monitor Process Performance . . . 16
- Assess Data Quality . . . 17
- Manage Methodology and Version Lifecycle . . . 17
- Audit and Compliance Reporting . . . 18
- Continuous Improvement . . . 18

Business Services and Interfaces . . . 19
- Boundary Interface Specifications . . . 19

Business Objects and Data Flow . . . 21
Business Events . . . 21
Framework Mappings . . . 21
Applying This to Your NSI . . . 22
- Maturity Assessment Checklist . . . 22
- End-to-End Scenarios . . . 24

VS-03.0 Data Editing and Enrichment

Purpose and Scope

This document specifies the complete business layer for the **Editing, derivation, and non-response correction** business domain, corresponding to the *VS-03.0 Data Editing and Enrichment Value Stream*. It also covers 4 governance process types:

- **Design** — how the editing, imputation, and derivation pipeline is specified and planned (produces metadata)
- **Implementation** — how the pipeline is built, configured, and tested (produces the execution process)
- **Execution** — how data is operationally edited, imputed, derived, and versioned (produces statistical data)
- **Management** — how the domain is monitored, quality-assessed, and improved (produces quality reports)

> **Out of Scope**
>
> Application and Technology layer specifications are out of scope for this document. Estimation, weighting, aggregation, and statistical modeling are out of scope — they belong to VS-04.0 Estimation, Analysis, and Statistical Modeling. VS-03 improves the quality of microdata; it does not produce estimates or aggregates.

A note on numbering: each `VS-0x.y` below is both a value stream stage (*what value* is added) and a business process (*how* it is done); the numbering maps one-to-one. See the key-concepts box above.

Position in the Statistical Value Chain

VS-03.0 is the third primary value stream — it receives Standardized Data from VS-02.0 and improves its quality and completeness so that it becomes trustworthy, analytically sound input for estimation and modeling. It is the "Edit & Enrich Information" orchestration stage where standardized data becomes high-quality statistical input.

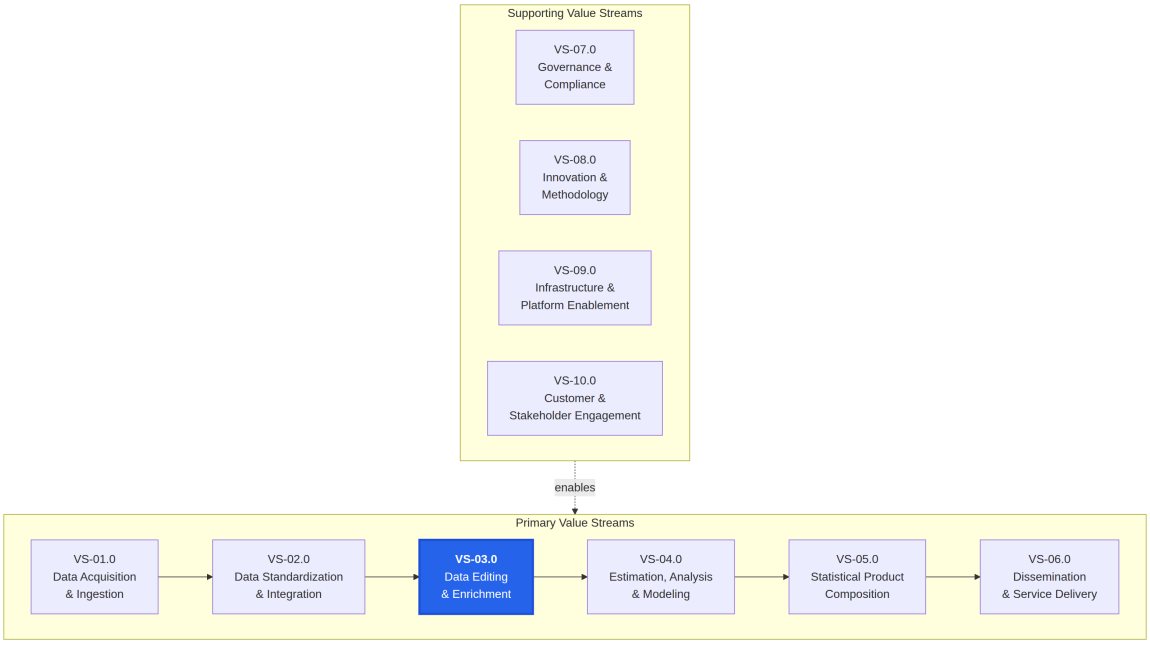


Figure 1: Value Stream Context Map. VS-03.0 is highlighted as the editing-and-enrichment stage that turns standardized data into analytically-ready processed data.

Steady State Transition

VS-03.0 transitions data from **Steady State #2 (Standardized Data)** — structurally aligned and standard-coded data, with values unchanged — to **Steady State #3 (Processed Data)** — data whose content has been validated, corrected, completed, and enriched, integrated with other datasets where applicable, with new derived variables and fully documented lineage.

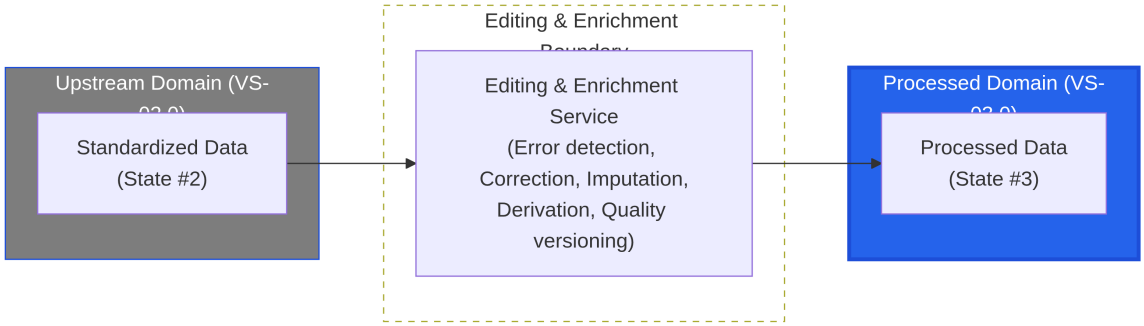


Figure 2: Steady State Transition from Standardized Data to Processed Data through the editing and enrichment service at the domain boundary.

Quality version criteria (Examples)*

- Logical errors, duplicates, outliers, and inconsistencies detected and resolved
- Invalid entries corrected per documented rules; every correction logged with the original value
- Missing and non-response values imputed using approved methods, with imputation flags
- Datasets integrated/linked with other sources where applicable
- New derived variables and indicators calculated and documented
- Inherited operations completed: interpretive coding, unit/currency conversion, reference-period alignment, value reconciliation
- A traceable quality version produced, with full edit lineage from each processed value back to its standardized source value

The editing discipline — values change, but never silently

Unlike VS-02 (which never alters a value), VS-03 **deliberately changes values** to improve quality. The guarantee here is not no-loss-of-content but **full traceability**: every edit, imputation, and derivation is rule-based, documented with a reason, reversible, and recorded so the original value is preserved alongside the processed value. Each change is attributable (who/what rule/when), and a versioned, reproducible Processed Data artefact is produced. The invariant is *no undocumented or irreversible change.*

Inherited hand-offs from VS-02.0

VS-02 deliberately left all value-changing work to this domain. VS-03 receives Standardized Data **plus** the metadata VS-02 attached — recorded unit/currency/reference-period bases, definition-difference records, and the list of unmapped/uncoded values — and performs, here, the operations VS-02 deferred:

- **Interpretive coding** of free-text values (e.g., free-text occupation to ISCO, the international occupation classification) — see §5.2
- **Unit and currency conversion** to a common basis — see §5.4
- **Reference-period alignment** (e.g., fiscal → calendar year) — see §5.4
- **Value reconciliation** across sources with differing definitions — see §5.2
- **Derivation** of new variables and indicators — see §5.4

The Four-Process Governance Pattern

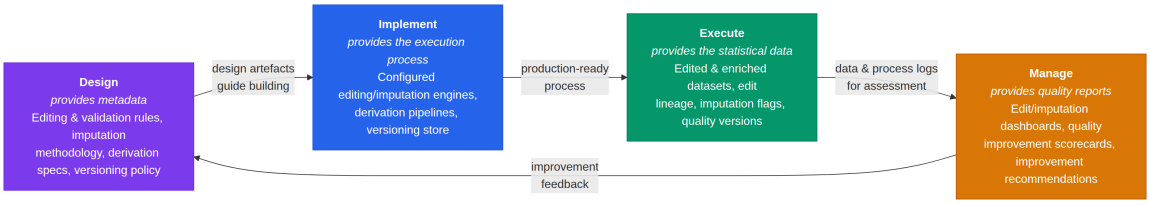


Figure 3: The Four-Process Governance Pattern. Each process type produces a distinct artefact class. The cycle closes when Management findings feed back into Design improvements.

Governing Principles

The following principles directly govern the VS-03.0 business domain:

Principle	Statement	Application to VS-03.0
SAF-P01	Actively participate in open source to maintain collective control over statistical processes	NSIs should contribute to open source communities for editing, imputation, and coding tooling (e.g., selective-editing engines, imputation libraries), ensuring collective control over critical methodological capabilities rather than vendor dependence.
SAF-P07	Implement loosely coupled processes and systems	The editing domain must be independent from upstream standardization and downstream estimation. Processed Data (State #3) serves as a decoupling interface — downstream consumers access edited, enriched microdata without knowledge of how it was corrected or imputed.
SAF-P08	Data is shared property	Processed data, once versioned, is available to all authorized consumers. Editing and enrichment should not be duplicated across statistical products when the same processed asset can serve multiple purposes.
SAF-P09	No data without metadata and classification	Every edit, imputation, and derived variable carries metadata: the rule applied, the original value, the imputation method and flag, the derivation formula, and a classification for derived variables. This is what makes the changes transparent and reproducible.

Principle	Statement	Application to VS-03.0
SAF-P10	Security By Design	Access controls, integrity verification, and a tamper-evident edit audit trail are embedded in the editing pipeline so that every change to a value remains attributable and traceable, not added as an afterthought.
SAF-P11	Privacy By Design	The domain works on pseudonymized microdata. Enrichment and integration with external datasets raise re-identification risk, so linkage is assessed for disclosure risk and minimization by design, not retrofitted.

i Cross-cutting principle application

Several principles cut across all four process types. SAF-P09 (no data without metadata and classification) is the backbone of the editing discipline: Design defines the rules and required edit metadata, Execute captures the original value, rule, and reason for every change, and Manage assesses edit quality and closes gaps. SAF-P10 (Security By Design) governs the tamper-evident edit audit trail across Execution (Sections 5.1–5.5). SAF-P11 (Privacy By Design) applies especially to integration/enrichment (Sections 3.4 and 5.4), where linkage can raise re-identification risk. SAF-P07 (loosely coupled) governs both the State #2 intake interface from VS-02.0 and the State #3 handover interface to VS-04.0.

Capability Foundations

VS-03.0 draws on the following SAF capabilities:

Capability Group	Capabilities	Role in VS-03.0
1.1 Statistical design and build	1.1.2 Design statistics product, 1.1.3 Build statistics product	Design the editing/imputation methodology and derivation specifications, and build the editing pipeline
2.1 Data management	2.1.1 Data lifecycle management, 2.1.2 Metadata management, 2.1.3 Statistical quality management, 2.1.4 Statistical register management	Manage data, edit metadata, and quality versions; assess quality improvement; integrate with statistical registers
3.2 Computerization and automation	3.2.1 Application development, 3.2.2 Functional management	Build and maintain editing, imputation, derivation, and integration engines
3.3 Security and Privacy	3.3.1 Privacy management, 3.3.2 Security planning	Manage disclosure risk in enrichment/linkage and secure the edit audit trail
4.1 Governance	4.1.1 Strategy and governance, 4.1.2 Policy and planning, 4.1.4 Improve management, 4.1.5 Accountability	Govern the domain, steward editing methodology, plan improvements, report on auditability and compliance

Metadata Perspective

It is considered that metadata is an essential product of the Design phase and a critical concern of every other phase. In this domain, metadata is what keeps value changes transparent: every edit, imputation, and derivation must be self-describing. The table below consolidates the metadata produced across all four process types, covering both data metadata (about the datasets) and process metadata or paradata (about how the process runs).

Process Type	Data Metadata Produced	Process Metadata (Paradata) Produced
Design	Editing/validation rule sets, imputation methodology, derivation formulae, integration/linkage rules, quality-version policy	Design decisions log, methodology version, review and approval records
Implement	Engine configurations, rule parameters, imputation model parameters, derivation definitions, versioning templates	Build log, test results, edit-impact test outcomes, deployment approvals
Execute	Per-record edit lineage (original value, rule, reason), imputation flags and methods, derived-variable formulae and inputs, quality-version identifiers	Operator identity, timing, volumes, edit/imputation rates, rule/model version, manual-override records
Manage	Quality-improvement scores vs. baseline, edit/imputation rate reports, lineage assessments, trend analyses	Edit dashboards, quality trends, improvement decisions, audit findings

Business Roles and Actors

Role	Process Orientation	Description
Upstream Data Provider (VS-02.0)	External (internal NSI)	Supplies Standardized Data (State #2) and its mapping/metadata to the editing domain
Methodology Authority	External	Sets the editing, imputation, and derivation methods the NSI conforms to (e.g., methodology unit, Eurostat guidance)
Editing Methodologist	Design	Designs error-detection and editing rules, selective-editing strategy, and quality criteria
Imputation Specialist	Design	Designs imputation methodology, models, donor strategies, and imputation flags
Derivation Designer	Design	Specifies derived variables and indicators, formulae, and required inputs
Data Engineer	Implementation	Builds and configures editing, imputation, derivation, and integration engines and the pipeline
System Configurator	Implementation	Configures editing services, reference/donor-data connections, and access controls
Test Manager	Implementation	Plans and executes testing of the editing pipeline (technical, edit-impact, reproducibility)
Process Owner	Management	Accountable for the end-to-end performance of the editing and enrichment domain
Compliance Officer	Management	Ensures edit auditability and disclosure-risk compliance, produces audit reports
Quality Manager	Management	Monitors quality-improvement KPIs, produces quality reports, and drives continuous improvement

Role	Process Orientation	Description
Subject-Matter Analyst	Execution	Reviews flagged records, adjudicates edits, and signs off domain-specific corrections
Coding Specialist	Execution	Performs interpretive coding of free-text values to standard classifications (handed over from VS-02)
Data Quality Analyst	Execution	Runs error detection, evaluates quality improvement, and logs edit lineage and exceptions
Data Steward	Execution	Registers processed datasets and quality versions, manages metadata entries, and applies access controls

Design Processes — "provides metadata"

Framework alignment: GSBPM Phases 1–2 — see Framework Mappings.

The design phase produces the **metadata artefacts** that govern all subsequent phases — the editing and validation rules, imputation methodology, derivation specifications, and the versioning policy. Without well-designed rules and methods, editing becomes ad-hoc and untraceable, and management has no baseline to measure quality improvement against.

Design Error-Detection and Validation Rules

The starting point of editing is defining what counts as an error. This sub-process establishes the validation and error-detection rules: logical/consistency checks (cross-field, intra-record, and against reference data), range and outlier-detection rules, duplicate-detection rules, and completeness checks. A **selective-editing** strategy is designed so that effort concentrates on errors that materially affect results — score functions and thresholds determine which records are auto-corrected, which are flagged for manual review, and which are left. The team designs the error taxonomy and the disposition options (auto-correct, flag for review, accept), aligned with GSBPM 2.5 (Design processing and analysis).

Activity	Key Outputs	Framework Ref
Define logical/consistency, range, and outlier detection rules	Validation Rule Set	GSBPM 2.5
Define duplicate- and completeness-detection rules	Validation Rule Set	GSBPM 2.5
Design selective-editing score functions and review thresholds	Selective-Editing Strategy	GSBPM 2.5
Define error taxonomy and disposition options (auto-correct / review / accept)	Error Taxonomy & Disposition Policy	GSBPM 2.5

NSI Example — Structural Business Statistics:

The SBS team designs consistency rules (turnover 0, employees 0, turnover-per-employee within a plausible band by NACE class), outlier rules using a Hidiroglou–Berthelot test on year-on-year ratios, and a selective-editing score that flags only units whose potential error would move a published aggregate by more than a set threshold. Everything else is accepted without manual touch.

Design Imputation Methodology

This sub-process designs how missing and non-response values are completed. The team selects imputation methods per variable and missingness type — deterministic/rule-based imputation, donor methods (hot-deck, nearest-neighbor), model-based methods (regression, ratio), and the use of external/administrative sources — and defines donor pools, matching variables, and acceptance constraints (e.g., edit rules the

imputed value must still satisfy). Crucially, the design mandates an **imputation flag** on every imputed value so that imputed and observed values are always distinguishable downstream, aligned with GSBPM 2.5.

Activity	Key Outputs	Framework Ref
Select imputation methods per variable and missingness type	Imputation Methodology Document	GSBPM 2.5
Define donor pools, matching variables, and acceptance constraints	Donor & Matching Specification	GSBPM 2.5
Specify use of external/administrative sources for imputation	External-Source Imputation Rules	GSBPM 2.5
Mandate imputation flags and method codes on every imputed value	Imputation Flagging Standard	SAF-P09

NSI Example — Labour Force Survey:

For item non-response on income, the LFS design specifies regression imputation using age, education (ISCED), occupation (ISCO), and region as predictors, with a constraint that imputed income remains within the valid range for the respondent's earnings bracket. Every imputed value receives a flag and a method code so analysts can exclude or down-weight imputed values if needed.

Design Derivation and Coding Specifications

This sub-process specifies the new variables and indicators to be derived, and the interpretive coding handed over from VS-02. For derivation, each derived variable is defined by a formula, its input variables, units, and the classification it belongs to (per SAF-P09). For interpretive coding, the design specifies the coding scheme (e.g., ISCO for occupation free text, COICOP for product names), the coding approach (manual, computer-assisted, or machine-learning-assisted with confidence thresholds), and the review workflow for low-confidence cases — the work VS-02 deferred because it requires judgement. The team also designs how unit/currency/reference-period conversions are computed from the bases VS-02 recorded as metadata, aligned with GSBPM 2.5.

Activity	Key Outputs	Framework Ref
Specify derived variables and indicators: formulae, inputs, units, classification	Derived-Variable Catalog	GSBPM 2.5
Design interpretive coding approach and review workflow (free-text → standard codes)	Coding Methodology	GSBPM 2.5
Design unit/currency/reference-period conversion rules from VS-02 metadata	Conversion Specification	GSBPM 2.5
Define metadata and classification for every derived variable	Derived-Variable Metadata Standard	SAF-P09

Design Integration and Linkage Rules

Where Processed Data is to be integrated with other datasets, this sub-process designs the linkage. It specifies the linkage keys (e.g., the statistical register identifier aligned in VS-02), deterministic and

probabilistic matching rules, conflict-resolution rules where linked sources disagree on a value, and the **value-reconciliation** policy for the definition differences VS-02 recorded as metadata. Because linkage can increase disclosure risk on pseudonymized microdata, a privacy/disclosure-risk assessment is designed per SAF-P11, aligned with GSBPM 5.1 design.

Activity	Key Outputs	Framework Ref
Define linkage keys and deterministic/probabilistic matching rules	Linkage Rule Set	GSBPM 2.5
Design conflict-resolution and value-reconciliation rules across sources	Reconciliation Policy	GSBPM 2.5
Design disclosure-risk assessment for integration/enrichment	Disclosure-Risk Assessment Design	SAF-P11
Define lineage requirements for linked/reconciled values	Linkage Lineage Specification	CSDA Traceability

Design Quality Evaluation, Versioning, and Audit Framework

The final design sub-process ensures that quality is measured and that every change is traceable. It defines the quality-evaluation criteria (improvement of completeness and accuracy against the State #2 baseline), the **edit audit-trail** requirements (original value, rule applied, reason, actor, timestamp for every change), the quality-versioning policy (how versions are identified, what triggers a new version, immutability and retention), and the role-based access controls for processed microdata in line with SAF-P10. Reproducibility is designed in: a quality version must be regenerable from the standardized input plus the recorded rules.

Activity	Key Outputs	Framework Ref
Define quality-evaluation criteria vs. the State #2 baseline	Quality Criteria Document	GAMSO QM
Define edit audit-trail requirements (original value, rule, reason, actor, time)	Edit Audit-Trail Specification	SAF-P10
Define quality-versioning policy: identity, triggers, immutability, retention	Versioning Policy	CSDA Traceability
Design role-based access control and reproducibility for processed microdata	Access Control & Reproducibility Design	SAF-P10

Implementation Processes — "provides the execution process"

Framework alignment: GSBPM Phase 3 (Build) — see Framework Mappings.

The implementation phase takes the metadata artefacts from Design and **builds, configures, and tests** the production-ready editing and enrichment pipeline. Nothing runs operationally until this phase is complete.

Build Error-Detection and Editing Engines

This sub-process turns the validation and editing rule designs into operational tools. Error-detection components are built to run the designed consistency, range, outlier, duplicate, and completeness checks and to compute the selective-editing scores. The editing engine is built or configured to apply auto-corrections per the designed rules and to route flagged records to the review workflow, capturing for every change

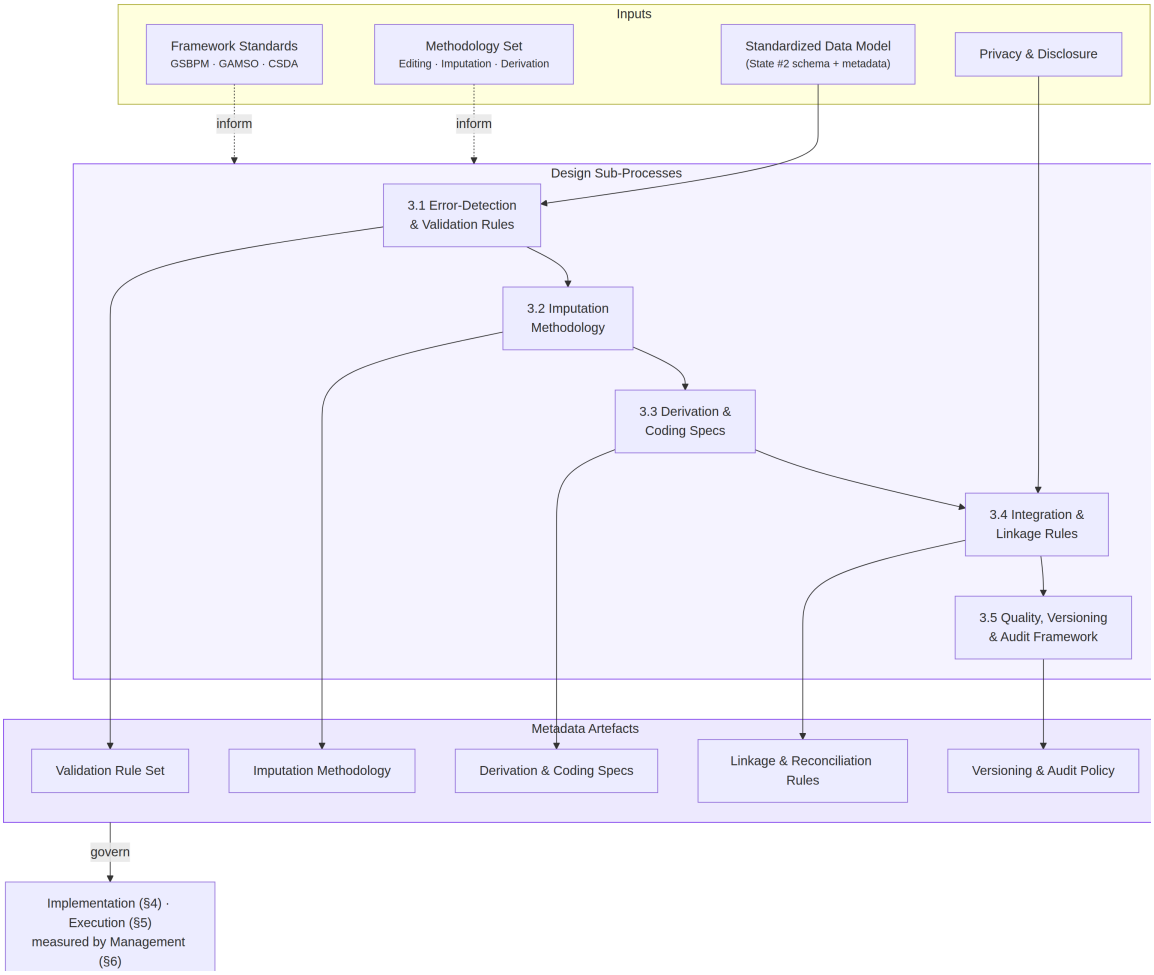


Figure 4: Design Process Decomposition. The standardized data model, methodology set, and framework standards flow into five design sub-processes, producing metadata artefacts that govern Implementation, Execution, and Management.

the original value, the rule applied, and a reason code. All editing logic is version-controlled in line with SAF-P10 so that any edit is reproducible and attributable, following GSBPM 3.1–3.2.

Activity	Key Outputs	Framework Ref
Build error-detection components (consistency, range, outlier, duplicate, completeness)	Production-ready detection module	GSBPM 3.2
Build/configure the editing engine (auto-correction + review routing)	Configured editing engine	GSBPM 3.2
Configure edit-lineage capture (original value, rule, reason, actor) for every change	Edit audit-trail capture	SAF-P10
Version-control all editing logic	Versioned editing repository	SAF-P10

NSI Example — Price Statistics:

The implementation team builds an outlier detector for daily price changes, an editing engine that applies designed corrections (e.g., decimal-point fixes), and a review queue for flagged price jumps. Every correction stores the original scraped price, the rule, and the analyst's adjudication, so the edit can be audited and reversed.

Configure Imputation and Coding Services

The designed imputation methodology and coding methodology are translated into automated services. Imputation services are configured for each method (deterministic, donor, model-based, external-source), with donor pools and matching variables loaded and acceptance constraints enforced, and every imputed value flagged with its method code, following GSBPM 3.2. The interpretive coding service handed over from VS-02 is configured — computer-assisted and/or ML-assisted coding for free-text values with the designed confidence thresholds, and a manual review workflow for low-confidence cases. Unit/currency/reference-period conversion routines are configured to compute values from the bases VS-02 recorded.

Activity	Key Outputs	Framework Ref
Configure imputation services per method, with donor pools and constraints	Configured imputation services	GSBPM 3.2
Enforce imputation flagging and method codes	Flagged imputation outputs	SAF-P09
Configure interpretive coding service (free-text → standard codes) with review workflow	Coding service + review queue	GSBPM 3.2
Configure unit/currency/reference-period conversion routines	Conversion service	GSBPM 3.4

Build Derivation, Integration, and Versioning Pipelines

This sub-process implements derivation, dataset integration, and quality versioning. Derived-variable calculators are built from the designed formulae, producing each derived variable with its metadata and classification. The integration/linkage component is configured with the designed keys and matching rules, applying the conflict-resolution and value-reconciliation policy and capturing linkage lineage. The quality-versioning store is built so that each accepted Processed Data version is immutable, identified, reproducible from inputs plus rules, and accompanied by its full edit lineage. Event notifications are configured so that downstream consumers (VS-04.0) are alerted when a processed quality version is registered.

Activity	Key Outputs	Framework Ref
Build derived-variable calculators from designed formulae	Derivation pipeline	GSBPM 3.2
Configure integration/linkage with conflict-resolution and reconciliation	Integration component	GSBPM 3.2
Build the quality-versioning store (immutable, identified, reproducible)	Versioning store	CSDA Traceability
Configure “Processed Data Registered” event for downstream consumers	Event publication	GSBPM 3.2

Test and Validate the Pipeline

Before the pipeline goes into production, it must be rigorously verified. Technical testing covers all individual components — detectors, editing engine, imputation and coding services, derivation calculators, integration component, and versioning store (GSBPM 3.5). Integration testing validates the end-to-end flow from standardized intake through editing, imputation, derivation, integration, to versioned registration. An **edit-impact test** confirms that edits and imputations move quality indicators in the intended direction without over-editing, and a **reproducibility test** confirms a quality version can be regenerated identically from the standardized input plus the recorded rules. Imputation is evaluated against a holdout/known-value sample. The phase concludes with finalization of the production system and documentation of operational procedures (GSBPM 3.6–3.7).

Activity	Key Outputs	Framework Ref
Technical testing of all individual components	Unit test reports	GSBPM 3.5
Integration testing: end-to-end flow from standardized intake to versioned registration	Integration test report	GSBPM 3.5
Edit-impact test (intended quality change, no over-editing)	Edit-impact report	GSBPM 3.6
Reproducibility test (regenerate a version from inputs + rules)	Reproducibility report	GSBPM 3.6
Evaluate imputation against a holdout/known-value sample	Imputation evaluation report	GSBPM 3.6
Finalize production system and document operational procedures	Operational Procedures Manual, Production-Ready Pipeline	GSBPM 3.7

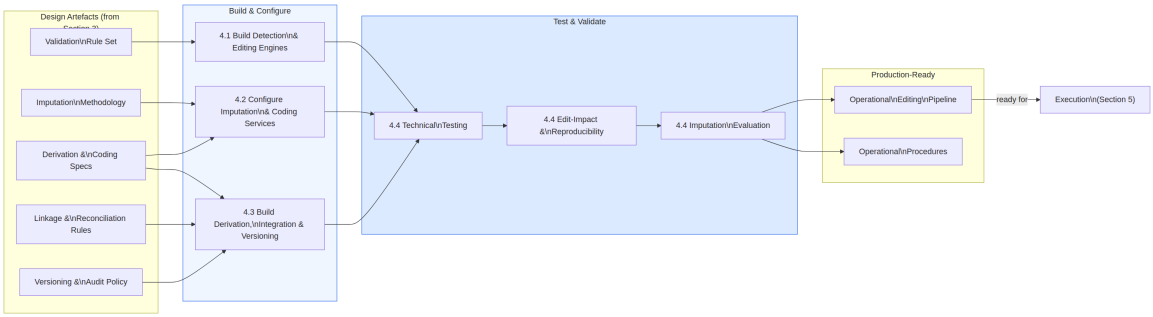


Figure 5: Implementation Pipeline from design artefacts through build/configure/test to production-ready system.

Execution Processes — "provides the statistical data"

Framework alignment: the later GSBPM phases for this domain — see Framework Mappings.

The execution phase runs the **designed and implemented pipeline** operationally for each production cycle. This corresponds to the five VS-03 value stream stages. In a mature steady state, much of the work has been pre-designed (Section 3) and pre-built (Section 4) — execution focuses on running the cycle, applying edits/imputations/derivations, and capturing the full edit lineage so that every value change is documented and reversible.

Conduct Error Detection and Quality Diagnostics

Value created: *A clear, prioritized picture of where the data is wrong, incomplete, or inconsistent.*

When a "Standardized Data Registered" event is received from VS-02.0, the editing domain begins by running the designed detection rules on the incoming dataset. Per-cycle execution applies the consistency, range, outlier, duplicate, and completeness checks, computes the selective-editing scores, and produces a diagnostic report: the location and type of each suspected error, its severity, and its disposition (auto-correct, flag for review, or accept). This corresponds to GSBPM 5.3 (Review and validate). No values are changed yet — this sub-process diagnoses; the correction happens in 5.2.

Activity	Key Outputs	Framework Ref
Run consistency, range, outlier, duplicate, and completeness checks	Error Diagnostic Report	GSBPM 5.3
Compute selective-editing scores and prioritize errors	Prioritized error list	GSBPM 5.3
Assign disposition (auto-correct / flag for review / accept)	Disposition list	GSBPM 5.3
Log all detected issues to the edit audit trail	Audit trail entries	SAF-P10

NSI Example — Structural Business Statistics:

The SBS file (Standardized Data registered by VS-02.0) is diagnosed: 312 units fail the turnover-per-employee plausibility band, 47 are duplicates, 1,204 have missing employee counts. Selective-editing scores prioritize 89 units whose potential error would materially affect a published aggregate; the remaining low-impact flags are scheduled for automatic treatment.

Perform Data Editing and Correction

Value created: *Errors corrected and inconsistencies resolved, with every change documented and reversible.*

This sub-process applies corrections to the detected errors, aligned with GSBPM 5.4 (Edit and impute). Auto-correction rules are applied to the records the design earmarked for automatic treatment; flagged records are routed to subject-matter analysts who adjudicate and correct them. For every change, the editing engine records the original value, the rule or analyst decision applied, a reason code, the actor, and a timestamp — so the edit is fully attributable and reversible. This sub-process also performs the **interpretive coding** handed over from VS-02 (coding free-text values such as occupation to ISCO, with low-confidence cases reviewed) and applies the designed **value-reconciliation** rules where integrated sources disagree on a value. No edit may leave a record violating the validation rules.

Activity	Key Outputs	Framework Ref
Apply auto-corrections per designed rules	Corrected records	GSBPM 5.4
Route flagged records to subject-matter review and adjudication	Adjudicated corrections	GSBPM 5.4
Perform interpretive coding of free-text values (e.g., occupation → ISCO)	Coded values + confidence/review records	GSBPM 5.2
Apply value-reconciliation rules across integrated sources	Reconciled values	GSBPM 5.4
Record original value, rule, reason, actor, and time for every change	Edit lineage records	SAF-P10

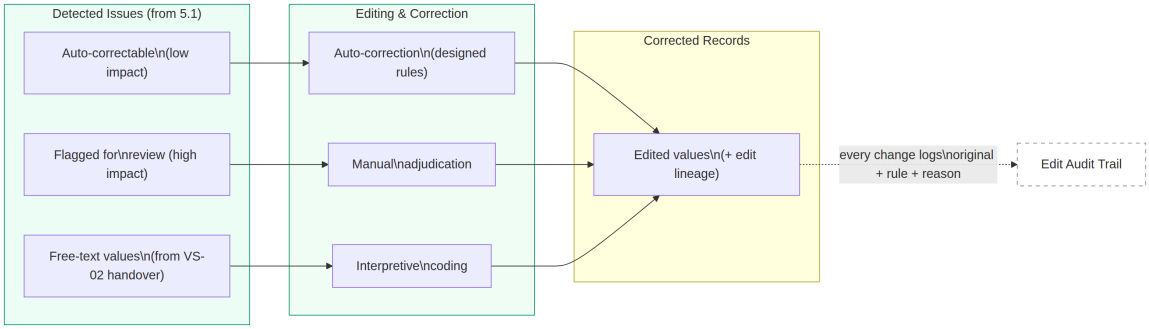


Figure 6: Editing and Correction Architecture. Auto-correctable, review-flagged, and free-text values are treated through automatic correction, manual adjudication, and interpretive coding respectively; every change writes original value, rule, and reason to the edit audit trail.

NSI Example — Labour Force Survey Coding:

The 42,000 free-text occupation strings handed over from VS-02 are coded to ISCO-08: 88% are auto-coded at or above the 0.85 confidence threshold, 5,040 go to computer-assisted coding, and 1,200 ambiguous cases to expert manual coding. Each coded value records the source text, the assigned code, the method, and the confidence — fully auditable as an editing decision.

Impute Missing or Non-Response Data

Value created: *Complete datasets where gaps are filled with the best available estimate, clearly flagged as imputed.*

This sub-process fills missing and non-response values using the designed imputation methods, aligned with GSBPM 5.4. The appropriate method is applied per variable and missingness pattern — deterministic, donor (hot-deck/nearest-neighbor), model-based, or external-source — subject to the acceptance constraints so that imputed values still satisfy the edit rules. Every imputed value receives an **imputation flag** and a method code so that imputed and observed values are always distinguishable downstream, and the imputation inputs are recorded in the lineage. Imputed values never overwrite observed values; they fill only genuine gaps.

Activity	Key Outputs	Framework Ref
Select and apply the designed imputation method per variable/missingness pattern	Imputed values	GSBPM 5.4

Activity	Key Outputs	Framework Ref
Enforce acceptance constraints (imputed values satisfy edit rules)	Constraint-checked imputations	GSBPM 5.4
Flag every imputed value with method code and record inputs	Imputation flags & lineage	SAF-P09
Log imputation actions to the edit audit trail	Audit trail entries	SAF-P10

NSI Example — Labour Force Survey:

Item non-response on income (about 9% of records) is imputed by the designed regression model using age, ISCED, ISCO, and region; each imputed income carries an `imputed=true` flag and method code `REG-01`, and the predictor values used are recorded so the imputation can be reproduced or revised.

Derive Additional Variables and Indicators

Value created: *Enriched datasets carrying the analytical variables downstream production needs.*

This sub-process calculates the derived variables and indicators specified in design, aligned with GSBPM 5.5 (Derive new variables and units). Each derived variable is computed from its documented formula and inputs, given its metadata and classification, and recorded with lineage to the inputs it was derived from. This sub-process also computes the **unit/currency/reference-period conversions** from the bases VS-02 recorded as metadata — for example converting turnover from thousands to units, or aligning a fiscal-year figure to the calendar year — each conversion documented with its rule and rate so the original value remains recoverable. Where datasets are to be combined, the integration/linkage component links records on the designed keys and the lineage records the linkage.

Activity	Key Outputs	Framework Ref
Calculate derived variables and indicators from designed formulae	Derived variables (+ metadata)	GSBPM 5.5
Compute unit/currency/reference-period conversions from VS-02 bases	Converted values (+ conversion lineage)	GSBPM 5.5
Integrate/link datasets on designed keys where applicable	Integrated dataset (+ linkage lineage)	GSBPM 5.1
Record derivation/conversion formulae and inputs as lineage	Derivation lineage	SAF-P09

NSI Example — Tax Register to Business Statistics:

Turnover, recorded by VS-02 as "thousands of EUR" on a fiscal-year basis, is converted here to units on a calendar-year basis using the designed conversion and apportionment rules; the conversion factor and the original value are stored in lineage. Turnover-per-employee is then derived as a new indicator with its formula and inputs recorded.

Perform Quality Evaluation and Versioning

Value created: *A trustworthy, traceable Processed Data version with measured quality improvement.*

The final execution sub-process evaluates the result and produces the official quality version, aligned with GSBPM 5.8 (Finalise data files). Quality is evaluated against the State #2 baseline — improvement in completeness and accuracy, edit and imputation rates, and the number of derived variables added — and the edit lineage is verified complete (every change attributable and reversible). A persistent, immutable

quality version of the Processed Data is created with a unique identifier, registered in the catalog per SAF-P09 with its full lineage, imputation flags, derived-variable metadata, and quality assessment, and access permissions are applied per SAF-P10. The sub-process concludes by publishing a "Processed Data Registered" event, notifying downstream VS-04.0 that new processed data is available for estimation and analysis.

Output: Processed Data (Steady State #3) — the official handover artefact to the rest of the statistical value chain.

Activity	Key Outputs	Framework Ref
Evaluate quality improvement vs. the State #2 baseline	Quality Evaluation Report	GSBPM 5.8
Verify edit lineage is complete and reversible	Lineage completeness check	SAF-P10
Create an immutable, identified quality version of Processed Data	Versioned Processed Data	CSDA Traceability
Register metadata: edit lineage, imputation flags, derived-variable metadata, quality assessment	Catalog registration	SAF-P09
Publish "Processed Data Registered" event to notify VS-04.0	Event notification	SAF-P07

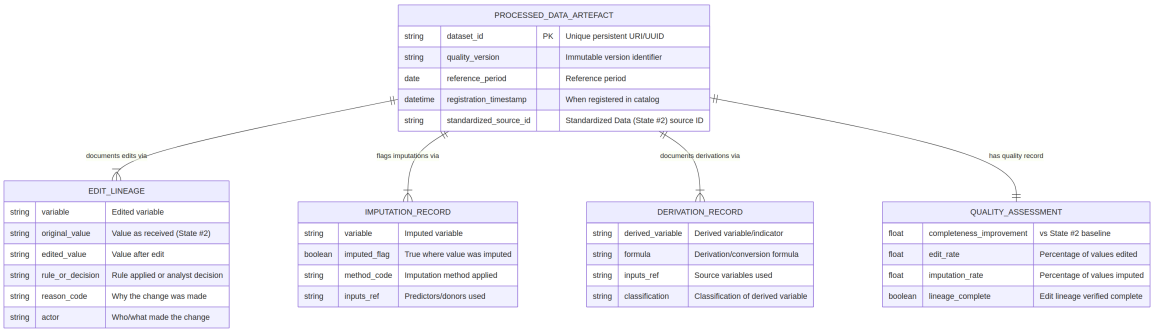


Figure 7: Processed Data Artefact Structure. Reading guide: each box is a type of record with its fields; a connecting line means the records are linked. Each processed quality version is linked to its edit lineage, imputation records, derivation records, and quality assessment, all traceable to the standardized source values.

Management Processes — "provides quality reports"

Framework alignment: GSBPM overarching Quality/Metadata Management — see Framework Mappings.

The management phase runs **alongside and after** execution. It monitors the editing domain's performance, assesses the quality improvement achieved, ensures every change is auditable, and feeds improvement recommendations back into the design phase — closing the governance cycle.

Monitor Process Performance

Operational performance of the editing domain must be tracked systematically across production cycles. The process owner and quality manager collect and aggregate key performance indicators, monitor throughput against processing windows, and track paradata — who edited what, when, with which rule or model version, how many manual interventions occurred. This information feeds process performance dashboards that give the process owner a real-time view of domain health, aligned with the GAMSO Quality Management capability.

Key Performance Indicators (KPIs):

KPI	Description	Measurement
Edit rate	Percentage of values changed by editing	Per variable, per cycle
Imputation rate	Percentage of values imputed	Per variable, per cycle
Manual-review rate	Percentage of records routed to manual adjudication	Per source, per cycle
Over-editing indicator	Share of edits with negligible impact on results	Per cycle
Coding completion rate	Percentage of free-text values successfully coded	Per classification, per cycle
Processing time	Duration from standardized intake to versioned registration	Per dataset, per cycle

Activity	Key Outputs	Framework Ref
Collect and aggregate KPIs per production cycle	Process Performance Dashboard	GAMSO QM
Monitor throughput against processing windows	Throughput Report	GAMSO QM
Track paradata: operator, timing, rule/model versions, interventions	Paradata Records	GAMSO QM
Generate process performance dashboards	Dashboard for Process Owner	GAMSO QM

Assess Data Quality

Beyond process performance, the quality improvement achieved must be evaluated against the criteria established in design. Each processed dataset is assessed across multiple quality dimensions: accuracy improvement (error rates before vs. after editing), completeness improvement (missingness before vs. after imputation), imputation quality (against holdout or known values), consistency (no residual rule violations), and coherence of derived variables. Results are compared against the designed quality criteria from Section 3.5. Quality indicators are produced per dataset, per source, and per cycle, and trends are tracked to detect over-editing, deteriorating sources, or imputation models that need recalibration. This aligns with the CSDA Data Quality capability and SAF Capability 2.1.3.

Activity	Key Outputs	Framework Ref
Evaluate accuracy and completeness improvement vs. baseline	Quality Improvement Report (per cycle)	SAF 2.1.3
Assess imputation quality against holdout/known values	Imputation Quality Report	CSDA Data Quality
Check for residual rule violations and over-editing	Consistency & Over-Editing Report	SAF 2.1.3
Identify quality trends across sources, variables, and cycles	Quality Trend Analysis	SAF 2.1.3

Manage Methodology and Version Lifecycle

The editing methodology and the quality versions it produces must remain current, complete, and well-governed. This sub-process governs the lifecycle of editing rules, imputation models, and derivation specifications — versioning them, managing transitions, and retiring obsolete rules. It verifies metadata completeness — that every processed dataset carries full edit lineage, imputation flags, and derived-variable

metadata as an SAF-P09 compliance check — and governs the quality-version store: retention, immutability, and reconciliation that catalog entries match stored versions. Metadata lineage from standardized through editing to processed state is tracked. As the CSDA emphasizes, complete, well-managed lineage is what makes edited data trustworthy and reusable.

Activity	Key Outputs	Framework Ref
Govern editing-rule, imputation-model, and derivation lifecycle and versions	Methodology Version Register	SAF-P09
Verify metadata completeness (edit lineage, imputation flags, derived metadata)	Metadata Quality Report	SAF-P09
Govern the quality-version store: retention, immutability, reconciliation	Version Store Audit	CSDA Traceability
Track metadata lineage from standardized to processed state	Metadata Lineage Audit	CSDA Traceability

Audit and Compliance Reporting

This sub-process produces the evidence that the editing domain operates within methodological, legal, and organizational requirements. Edit-audit reports demonstrate that every value change is attributable and reversible — original value, rule, reason, actor, and time recorded — supporting reproducibility and methodological transparency. Privacy/disclosure-risk reporting covers integration and enrichment, confirming that linkage on pseudonymized microdata stays within the assessed re-identification risk per SAF-P11. Methodology compliance reporting confirms that approved editing and imputation methods were applied, and security compliance reporting verifies that designed access controls and the tamper-evident audit trail have been applied. These reports satisfy the SAF Capability 4.1.5 (Accountability) obligation and feed into VS-07.0 Governance and Compliance.

Activity	Key Outputs	Framework Ref
Produce edit-audit reports: every change attributable and reversible	Edit Audit Summary	SAF 4.1.5
Report on disclosure-risk compliance for integration/enrichment	Disclosure-Risk Report	SAF-P11
Report on methodology compliance (approved editing/imputation methods)	Methodology Compliance Report	SAF 4.1.5
Report on security compliance: access controls, audit-trail integrity	Compliance Report (per cycle)	SAF-P10

Continuous Improvement

The final management sub-process closes the governance loop by analyzing performance and quality data to identify and drive improvements. Quality trends across production cycles are examined: are error rates falling, is over-editing decreasing, is imputation quality improving, are coding completion rates rising? Improvement opportunities are identified — refining edit rules that generate false positives, recalibrating imputation models, automating manual edits that recur, improving the coding service for free text, and tuning selective-editing thresholds to reduce over-editing. These improvement recommendations are fed back to the design processes (Section 3), prioritized based on impact and feasibility, and tracked through subsequent design-implement-execute cycles. This is the SAF Capability 4.1.4 (Improve management) in action, and it connects to the GAMSO concept of transferring mature capabilities from innovation into production.

Activity	Key Outputs	Framework Ref
Analyze quality trends across production cycles	Improvement Recommendations Report	SAF 4.1.4
Identify improvement opportunities: rule refinement, model recalibration, threshold tuning	Updated Design Inputs	SAF 4.1.4
Feed recommendations back to design processes (Section 3)	Design feedback (closing the governance loop)	SAF 4.1.4
Prioritize improvements based on impact and feasibility	Improvement Tracking Log	GAMSO Capability Mgmt
Track implementation of accepted improvements	Improvement Tracking Log	GAMSO Capability Mgmt

Business Services and Interfaces

The VS-03.0 business domain exposes the following services:

Service	Direction	Consumers	Mapped Processes
Error-Detection Service	Internal (VS-02.0 → NSI)	Standardized data consumers, editing teams	5.1 Conduct Error Detection
Editing and Correction Service	Internal	Editing teams, subject-matter analysts	5.2 Perform Data Editing and Correction
Imputation Service	Internal	Editing teams, methodologists	5.3 Impute Missing or Non-Response Data
Derivation and Integration Service	Internal	Editing teams, downstream analysts	5.4 Derive Additional Variables
Processed-Data Provisioning Service	Internal (NSI → VS-04.0)	Estimation & modeling teams, data analysts	5.5 Perform Quality Evaluation and Versioning
Quality-Version Catalog Service	Internal	All data consumers across the NSI	6.3 Manage Methodology & Version Lifecycle
Design and Methodology Service	Internal	Methodologists, editing/imputation designers	3.1–3.5 All design processes

Interface specifications:

- **Processed-Data Provisioning Service** interface: event-driven notification ("Processed Data Registered") + catalog query API for downstream consumers to discover and access processed quality versions and their edit lineage
- **Editing and Correction Service** interface: review/adjudication workbench for analysts + edit-audit query API
- **Quality-Version Catalog Service** interface: search/browse API for quality versions, edit lineage, imputation flags, and derived-variable metadata

Boundary Interface Specifications

The domain is loosely coupled (SAF-P07) to its neighbors through two steady-state interfaces. Specifying each boundary explicitly — the handover artefact, the metadata it must carry, the event that triggers it, and where responsibility transfers — makes the loose-coupling contract concrete and testable.

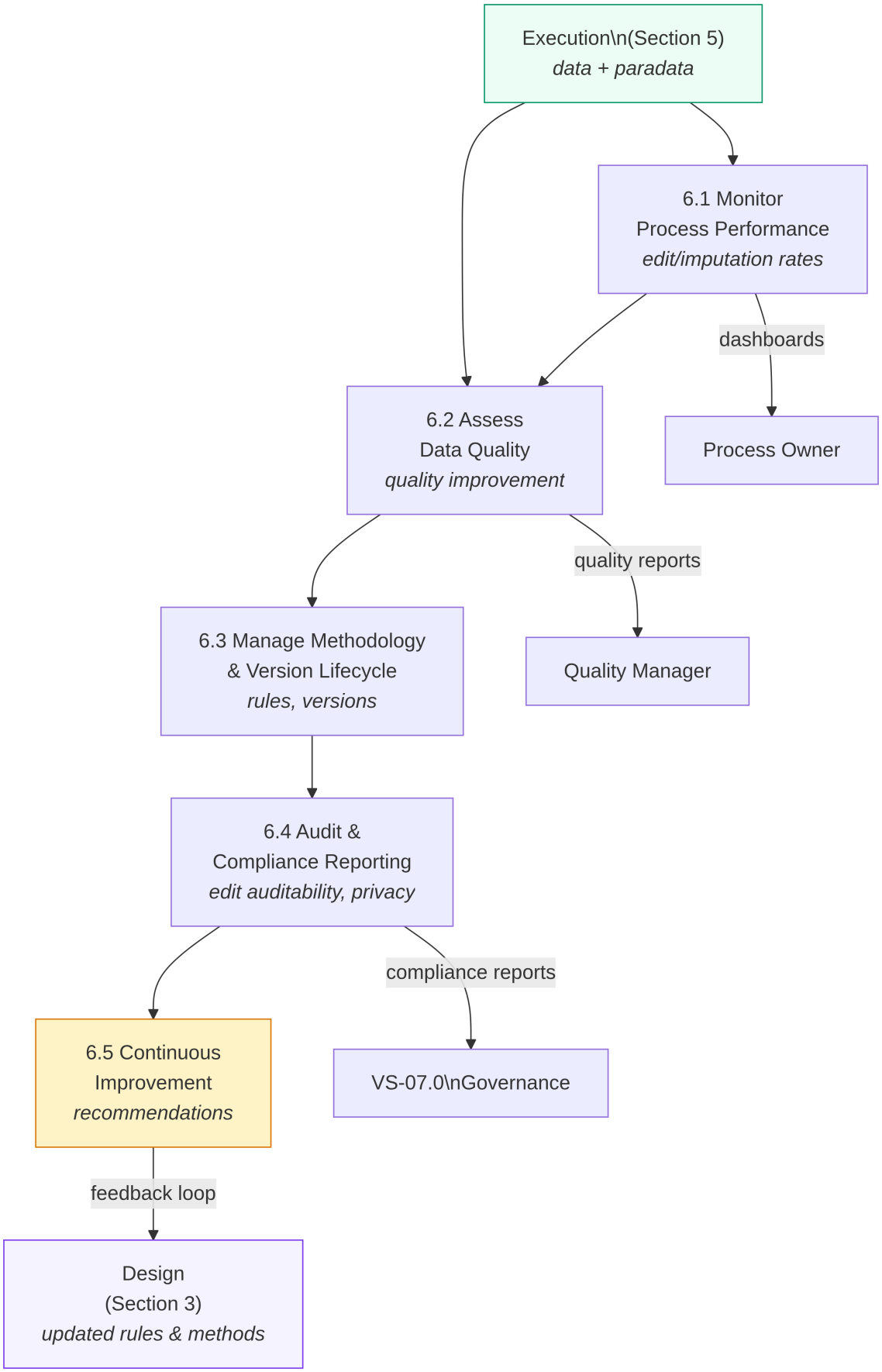


Figure 8: Management Feedback Loop. Execution data flows through monitoring, quality assessment, methodology/version management, and audit processes. Continuous improvement feeds recommendations back to Design, closing the governance cycle.

Boundary	Handover Artefact	Required Metadata	Trigger Event	Responsibility Transfer
VS-02.0 → VS-03.0 (intake)	Standardized Data (State #2)	Source-to-target mapping lineage, applied classification versions, recorded unit/currency/reference-period bases, list of unmapped/uncoded values for coding, definition-difference records, BIV classification	"Standardized Data Registered" event	VS-02.0 retains custody of the standardized artefact; VS-03.0 assumes responsibility for editing, imputation, derivation, interpretive coding, and value conversion on event receipt
VS-03.0 → VS-04.0 (handover)	Processed Data (State #3)	Edit lineage (original value, rule, reason), imputation flags and methods, derived-variable formulae and classification, integration/linkage lineage, quality-improvement assessment, quality-version identifier, BIV classification	"Processed Data Registered" event	VS-03.0 retains custody of the processed version; VS-04.0 assumes responsibility for estimation, weighting, and analysis on event receipt

Each interface exchanges a registered, immutable steady-state artefact plus its metadata — never internal working data — so neither side needs knowledge of the other's internal processing.

Business Objects and Data Flow

Business Events

Framework Mappings

This reference table maps each process group of this document to the international frameworks (GSBPM, GAMSO, CSDA) and the SAF capability model. It is intended for architects and auditors; the narrative sections above do not depend on it.

Process group	GSBPM	GAMSO	CSDA	SAF Capabilities
Design	Phase 1 (Specify Needs), Phase 2 (Design: 2.1–2.5)	Production (design activities), Capability Management	Data Governance, Metadata Management, Data Quality	1.1.2 Design statistics product, 2.1.2 Metadata management, 2.1.3 Statistical quality management

Process group	GSBPM	GAMSO	CSDA	SAF Capabilities
Implementation	Phase 3 (Build: 3.1–3.7)	Capability Management	Data Processing (setup), Data Governance (configuration)	1.1.3 Build statistics product, 3.2.1 Application development, 3.2.2 Functional management
Execution	Phase 5 (Process: 5.3 Review & validate, 5.4 Edit & impute, 5.5 Derive new variables, 5.8 Finalise data files; 5.1 Integrate)	Production (execution)	Data Processing (execution)	2.1.2 Metadata management, 2.1.3 Statistical quality management
Management	Overarching processes (Quality Management, Metadata Management)	Corporate Support (Quality Management, Data Management, Metadata Management), Capability Management (improvement)	Data Quality, Data Governance, Traceability	2.1.3 Statistical quality management, 4.1.4 Improve management, 4.1.5 Accountability

Applying This to Your NSI

Maturity Assessment Checklist

Use this checklist to assess your NSI's maturity across all four process types — not just execution.

Design Maturity

- ☐ Do we have documented error-detection and editing rules with a selective-editing strategy?
- ☐ Do we have a formal imputation methodology with mandated imputation flags?
- ☐ Do we have documented derivation specifications and a coding methodology for free-text values?
- ☐ Do we have integration/linkage rules with a disclosure-risk assessment design?
- ☐ Do we have a quality-versioning and edit-audit-trail policy defined before editing begins?

Implementation Maturity

- ☐ Are our editing rules built from design specifications (not ad-hoc fixes)?
- ☐ Do our imputation services flag every imputed value with its method?
- ☐ Is edit-lineage capture (original value, rule, reason, actor) automated for every change?
- ☐ Can a quality version be reproduced from the standardized input plus the recorded rules?
- ☐ Do we have a formal testing process (technical, edit-impact, reproducibility, imputation evaluation)?

Execution Maturity

- ☐ Do we run error detection and selective-editing diagnostics each cycle?
- ☐ Do we record the original value, rule, and reason for every edit?
- ☐ Do we flag imputed values and never overwrite observed values?
- ☐ Do we document the formula and inputs for every derived variable and conversion?
- ☐ Do we register every processed dataset as an immutable, identified quality version?

Management Maturity

- ☐ Do we track KPIs (edit rate, imputation rate, over-editing) across cycles?
- ☐ Do we evaluate quality improvement against the State #2 baseline?
- ☐ Do we govern editing-rule and imputation-model versions and the quality-version store?

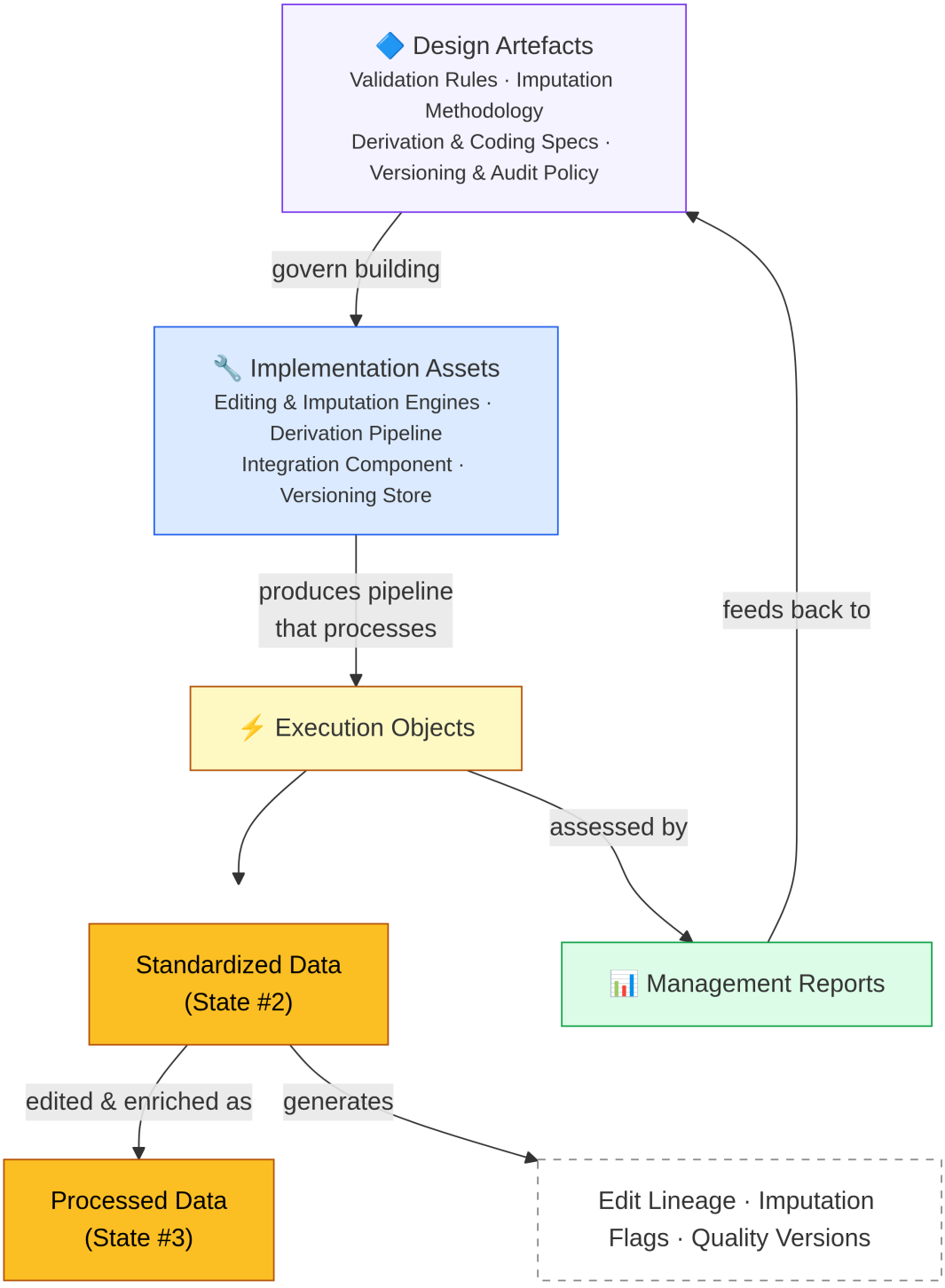


Figure 9: Business Object Lifecycle across all four process types. Design objects govern implementation, which produces the pipeline that edits and enriches execution objects, which are assessed by management objects, which feed back to design.



Figure 10: Event-Driven Choreography between the upstream domain (VS-02.0) and NSI teams, showing events across the Design → Implement → Execute → Manage cycle. The management phase closes the loop by feeding improvements back to design.

☐ Do we produce edit-audit, disclosure-risk, and methodology-compliance reports?
☐ Do we have a formal improvement process that feeds back into design?

End-to-End Scenarios

Each scenario walks through **all four process types**, demonstrating how the governance pattern works in practice.

Scenario A: Labour Force Survey (Coding, Editing, and Imputation)

Design Phase: Editing methodologists define consistency rules (e.g., age vs. labour-market status, hours vs. employment status) and a selective-editing strategy. Imputation specialists design regression imputation for item non-response on income, with constraints keeping imputed income in the respondent's plausible band. A coding methodology is designed for the free-text occupation strings handed over from VS-02 (ISCO-08, computer-assisted with a 0.85 confidence threshold, expert review below it). The versioning policy and edit audit-trail requirements are set.

Implementation Phase: Data engineers build the consistency-check and editing engine, configure the ISCO-08 coding service with its review queue, configure the income imputation model, and stand up the quality-version store. The pipeline is tested on a synthetic sample; the edit-impact test confirms edits move quality in the intended direction, the reproducibility test regenerates an identical version, and imputation is evaluated against a holdout sample (RMSE within target).

Execution Phase: For cycle Q3-2026, a Standardized Data Registered event arrives for 42,000 LFS returns. Error detection flags inconsistencies and outliers; auto-corrections are applied and high-impact records adjudicated. The 42,000 free-text occupations are coded to ISCO-08 (88% auto-coded, the rest computer-assisted and manually reviewed). Income non-response (9%) is imputed and flagged. Employment-status is harmonized to the ILO definition per the reconciliation rule. A quality version is created with full edit lineage and registered; VS-04.0 is notified.

Management Phase: Edit rate is 6.2%, imputation rate 9%, over-editing indicator low (selective editing working). Coding completion is 99.7%; a holdout re-check shows imputation RMSE stable. Improvement recommendations: refine two consistency rules generating false positives; retrain the occupation coder with this cycle's reviewed cases. Recommendations fed back to design.

Scenario B: Tax Register to Business Statistics (Editing, Conversion, Integration, Derivation)

Design Phase: The team designs outlier and consistency rules for turnover and employment, a value-reconciliation rule for the definition difference VS-02 recorded between the survey and VAT register, and conversion rules to turn turnover (recorded by VS-02 as "thousands of EUR" on a fiscal-year basis) into units on a calendar-year basis. Derived indicators (turnover-per-employee, productivity index) are specified with formulae and classification, and linkage rules join the tax data to the structural business survey on the SBR key.

Implementation Phase: Engineers build the editing engine, the unit/period conversion routines, the integration component (SBR-key linkage with conflict resolution), and the derivation calculators. Edit-impact and reproducibility tests pass; the conversion routine is checked so the original value remains recoverable from lineage.

Execution Phase: Q3-2026: a Standardized Data Registered event arrives for 2,347,891 tax records. Outliers and duplicates are detected and corrected with full lineage. Turnover is converted to units on the calendar year (conversion factor and original value stored), the tax and survey sources are integrated on the SBR key with the reconciliation rule applied where they disagree, and turnover-per-employee is derived. A quality version `PD-BUS-2026Q3-001` is created with lineage to `SD-BUS-2026Q3-001`, and VS-04.0 is notified.

Management Phase: Edit rate 1.8%, conversion applied to 100% of turnover with original values preserved, reconciliation applied to 12,400 linked units. No-residual-violation check passes. Disclosure-risk report confirms the linkage stays within assessed bounds. Improvement recommendation: tighten the outlier band for one NACE class; document a recurring reconciliation case as a standing rule. Recommendations fed back to design.

Scenario C: Web-Scraped Price Data (Coding, Imputation, Derivation)

Design Phase: The team designs the product classification (COICOP-2018) handed over from VS-02 for the free-text product names that had no source code, including an ML-assisted classifier with a confidence threshold and a review workflow. Currency-to-EUR and unit-to-standard-quantity conversion rules are designed from the bases VS-02 recorded, an imputation rule fills gaps where a product was not scraped on a given day (carry-forward with a staleness limit), and a daily price-index input is derived.

Implementation Phase: Engineers configure the COICOP classifier and review queue, the currency/unit conversion routines, the carry-forward imputation with its staleness flag, and the derivation of the index input. The classifier is evaluated against a manually-classified sample (90% agreement); below-threshold products route to review.

Execution Phase: For 2026-03-15, a Standardized Data Registered event arrives for 11,875 records. Products without a source code are classified to COICOP (95% above threshold, 5% reviewed). Prices are converted to EUR and standard units (original values preserved in lineage). Missing daily prices are imputed by carry-forward and flagged with staleness. The daily index input is derived. A quality version is created with full lineage and registered; VS-04.0 is notified.

Management Phase: Coding completion 99.2%, imputation rate (carry-forward) 4.8% with staleness within limits, conversion applied to all prices with originals preserved. A holdout check on classification shows 93% accuracy. Improvement recommendations: retrain the classifier for one recurring misclassified category; tighten the carry-forward staleness limit. Recommendations fed back to design.